

Node.js

What is Node.js

Node.js = running JavaScript outside the browser.

since normally JS runs in Chrome With Node.js JavaScript runs on your computer as a **server**.

What is Express

Express.js = a tool to build servers easily Without Express, making a server is hard.

first when opening a project we need to install them.

```
npm init -y
npm install express
```

Code Example.

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send("Hello World");
});

app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

Open browser → <http://localhost:3000> output should be "Hello World".

Code Explanation

```
const express = require('express');
```

`require('express')` → "give me the **Express.js library".

`const express = ...` → store it in a variable called `express`.

it's like saying Import the tool I installed with `npm install express`.

```
const app = express();
```

We are **creating your server here** `express()` returns an object (your app).

like saying Create my web server and call it `app`.

```
app.get('/', (req, res) => {  
  res.send("Hello World");  
});
```

```
(req, res) => {}
```

This is a function.

- `req` = request (what the user sends).
- `res` = response (what you send back).

◆ `res.send("Hello World")`

👉 This sends a response to the browser.

```
app.listen(3000, () => {  
  console.log("Server running");  
});
```

Starts the server at Port 3000

♦ `() => { console.log(...) }` This runs when server starts

So in terminal you'll see: Server running.

Catalog Service ***

```
const express = require('express');
const app = express();
app.use(express.json());

let books = [
  { id: 1, title: "How to get a good grade in DOS in 40 minutes a day", topic: "distributed systems", price: 30, stock: 5 },
  { id: 2, title: "RPCs for Noobs", topic: "distributed systems", price: 50, stock: 3 },
  { id: 3, title: "Xen and the Art of Surviving Undergraduate School", topic: "undergraduate school", price: 40, stock: 4 },
  { id: 4, title: "Cooking for the Impatient Undergrad", topic: "undergraduate school", price: 20, stock: 2 }
];

app.get('/search/:topic', (req, res) => {
  const topic = req.params.topic;
  const result = books.filter(b => b.topic === topic);
  res.json(result);
});

app.get('/info/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const book = books.find(b => b.id === id);
  res.json(book);
});
```

```

app.post('/update/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const change = req.body.change;
  const book = books.find(b => b.id === id);
  if (book) {
    book.stock += change;
    res.json({ message: "Updated successfully", book });
  } else {
    res.status(404).json({ error: "Book not found" });
  }
});

app.listen(3001, () => {
  console.log("Catalog Service running on http://localhost:3001");
});

```

the first two lines we already explained them.

```

app.use(express.json());

```

This tells the server → If someone sends JSON data, automatically convert it into a JavaScript object.

```

let books = [
  { id: 1, title: "DOS Guide", topic: "distributed systems",
    price: 30, stock: 5 },
  { id: 2, title: "RPCs for Noobs", topic: "distributed systems",
    price: 50, stock: 3 },
  { id: 3, title: "Xen Survival", topic: "undergraduate", price: 40,
    stock: 4 },
  { id: 4, title: "Cooking Guide", topic: "undergraduate", price: 20,
    stock: 2 }
];

```

Fake Database

SEARCH endpoint

```
app.get('/search/:topic', (req, res) => {  
  const topic = req.params.topic;  
  const result = books.filter(b => b.topic === topic);  
  res.json(result);  
});
```

this how we test it → <http://localhost:3001/search/distributed%20systems> ,%20 represents "space".

```
const topic = req.params.topic;
```

this line means Gets the value from URL so if the URL is `/search/distributed systems` that means

topic = "distributed systems".

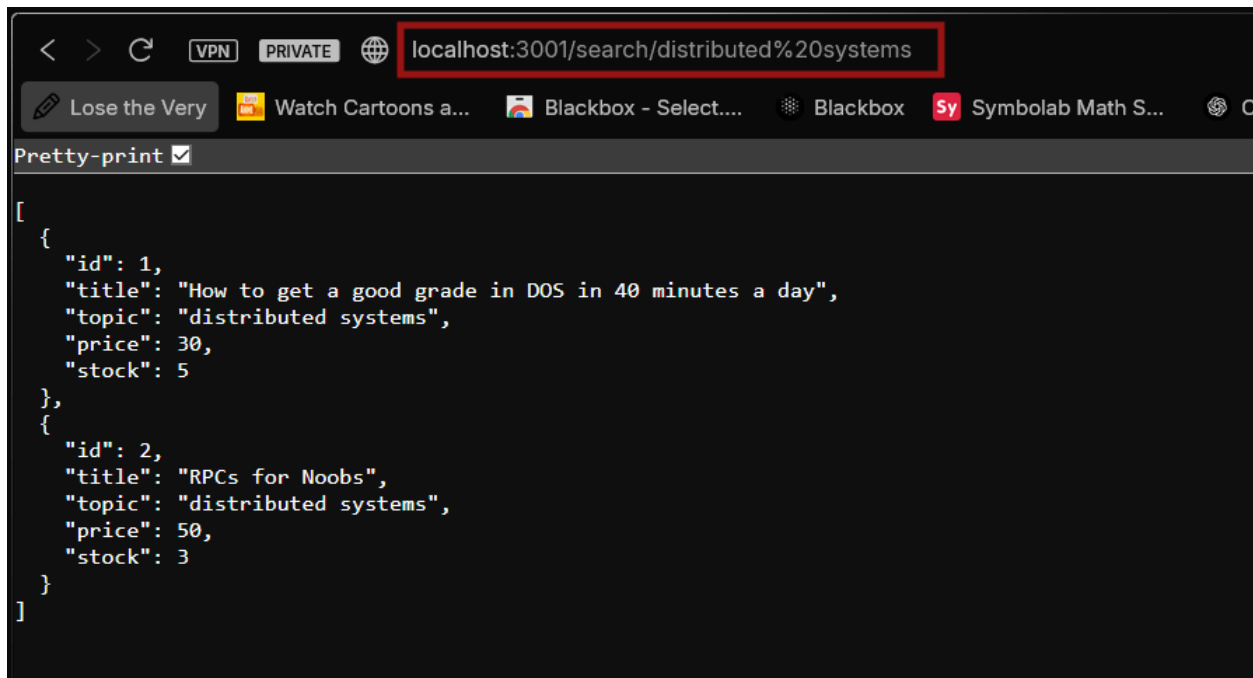
```
const result = books.filter(b => b.topic === topic);
```

Loop through all books and keep only matching ones.

```
res.json(result);
```

Send result as JSON to the client

now let's test it.



INFO endpoint

```
app.get('/info/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const book = books.find(b => b.id === id);
  res.json(book);
});
```

Route → `/info/2`

```
const id = parseInt(req.params.id);
```

Get ID from URL

- URL gives string `"2"`
- `parseInt` converts it → number `2`

UPDATE STOCK endpoint

```
app.post('/update/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const change = req.body.change;
  const book = books.find(b => b.id === id);
  if (book) {
    book.stock += change;
    res.json({ message: "Updated successfully", book });
  } else {
    res.status(404).json({ error: "Book not found" });
  }
});
```

This is difference here →

- `POST` = used to **change data**
- Not just read like `GET`

```
const change = req.body.change;
```

Get data from request body

Example request:

{ "change": -1} so change = -1

```
if (book) {
  book.stock += change;
  res.json({ message: "Updated successfully", book });
} else {
  res.status(404).json({ error: "Book not found" });
}
```

if the book does exist Update stock.

- `change = -1` → reduce stock
- `change = +1` → increase stock

Now finally to start the server you should type this in the Command prompt (**node index.js**).

```
PS C:\Users\Hp\OneDrive\Desktop\BazarProject\catalog-service>
node index.js
Catalog Service running on http://localhost:3001
```

Frontend Service ***

```
const express = require('express');
const axios = require('axios');
const app = express();
app.use(express.json());

const CATALOG_URL = 'http://localhost:3001';

app.get('/search/:topic', async (req, res) => {
  const topic = req.params.topic;
  try {
    const response = await axios.get(`${CATALOG_URL}/search/${topic}`);
    const books = response.data;
    const result = books.map(b => ({ id: b.id, title: b.title }));
    res.json(result);
  } catch (error) {
    res.status(500).json({ error: "Could not reach catalog service" });
  }
});

app.get('/info/:id', async (req, res) => {
  const id = req.params.id;
```



```

    try {
      const response = await axios.get(`${CATALOG_URL}/info/${id}`);
      res.json(response.data);
    } catch (error) {
      res.status(500).json({ error: "Could not reach catalog service" });
    }
  });

app.listen(3000, () => {
  console.log("Frontend Service running on http://localhost:3000");
});

```

◆ axios (NEW)

"A tool that lets your server send requests to other servers"

So instead of browser visiting URLs, your **server is visiting other servers**

```
const CATALOG_URL = 'http://localhost:3001';
```

This is where the Catalog Service lives So instead of repeating it every time.

```

app.get('/search/:topic', async (req, res) => {
  const topic = req.params.topic;
  try {
    const response = await axios.get(`${CATALOG_URL}/search/${topic}`);
    const books = response.data;
    const result = books.map(b => ({ id: b.id, title: b.title }));
    res.json(result);
  } catch (error) {
    res.status(500).json({ error: "Could not reach catalog service" });
  }
});

```

```
    rvice" });  
  }  
});
```

NEW THING: **async**

Means → "This function will wait for something (like a server response)"

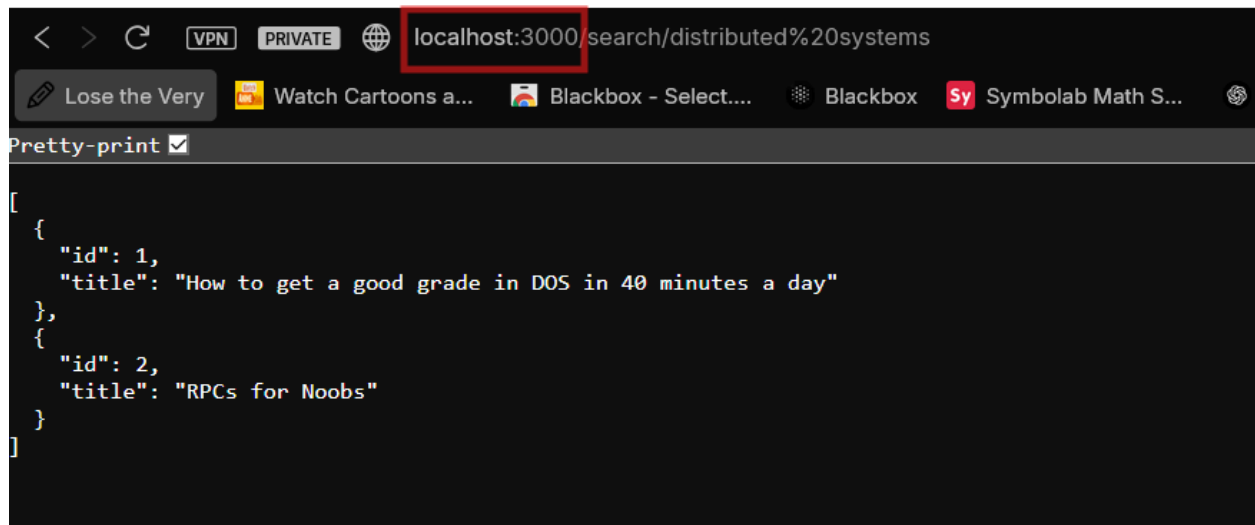
so basically in this frontend server if someone requests search/topic in it. the frontend server will go request it from the **catalog server**.

```
const response = await axios.get(`${CATALOG_URL}/search/${topic}`);
```

- `axios.get(...)` → send HTTP request
- `${CATALOG_URL}/search/${topic}` → [http://localhost:3001/search/distributed systems](http://localhost:3001/search/distributed%20systems)

Wait until response comes back.

testing it.



Docker Tutorial :

Docker (Lecture examples and explanation).